

LIBRARY MANAGEMENT SYSTEM

Software Technical Documentation Report

Project Title	Library Management System
Developer	Khizar Muzzamil & Ameer Muawiya Khattak
Language	C++ (ISO C++17)
IDE / Toolchain	GCC/G++, any POSIX IDE (VS Code, CLion, Code::Blocks)
Project Type	Library Management System (4th Semester) LAB
Version	1.0.0
Date	June 2026

Table of Contents

1. Project Overview
2. Features
3. Software Requirements
4. Project Architecture
5. Program Flow
6. Data Structures Used
7. Class Documentation
8. Function Documentation
9. Logic Explanation
10. Code Walkthrough
11. Object-Oriented Programming Concepts
12. Algorithms Used
13. Error Handling
14. Memory Management
15. User Interface
16. Diagram Suggestions
17. Screenshots Section
18. Code Quality Review
19. Optimization Suggestions
20. Conclusion

1. Project Overview

The **Library Management System (LMS)** is a console-based C++ application that automates the core operational processes of a book library. It provides librarians and system users with the ability to manage an inventory of books — adding, searching, updating, deleting, issuing, and returning them — through a structured, menu-driven interface. The system acts as a practical showcase of four fundamental data structures: a **Singly Linked List**, a **Binary Search Tree (BST)**, a **Queue**, and a **Stack** — each fulfilling a distinct and purposeful role within the system.

1.1 Problem Solved

Manual library record-keeping is error-prone, slow, and offers no intelligent search or undo capability. This system replaces that with an in-memory solution that offers $O(\log N)$ book retrieval via BST indexing, FIFO waitlist management for out-of-stock books, and a stack-driven undo mechanism for the two most destructive operations (delete and issue).

1.2 Target Users

- Library administrators performing daily book management.
- Students issuing or returning books.
- University evaluators assessing data structures implementation competency.

1.3 Key Objectives

- Implement a dual-storage model (Linked List + BST) for efficient CRUD and search.
- Apply Queue semantics to manage waiting students when books are unavailable.
- Implement an undo mechanism using a Stack to allow reversal of delete and issue operations.
- Ensure safe memory management through structured destructors and pointer cleanup.
- Provide robust input validation to prevent invalid data from corrupting the system state.

2. Features

Feature	Description
Add Book	Inserts a new Book record into the Linked List ($O(1)$ front insertion) and simultaneously indexes it in the BST by ID. Duplicate ID detection is performed before any insertion.
Display All Books	Traverses the entire Linked List from head to tail and renders a formatted tabular display of all fields (ID, Title, Author, Category, Total, Available), with column widths managed via <code>std::setw</code> .
Search by ID (BST)	Performs a recursive BST traversal to locate a book by its numeric ID in $O(\log N)$ average time. Outputs full details on success.
Search by Name (Linked List)	Performs a linear scan of the Linked List matching the full book title string. Time complexity $O(N)$. Reports first match found.
Update Book	Looks up a book by ID via BST. Presents the current record and allows the user to selectively update the Title and/or Author, leaving other fields unchanged if the user presses Enter.
Delete Book	Saves the complete book record to the undo Stack, then removes the node from both the BST and the Linked List atomically. Leaves the system consistent across both data structures.

Issue Book	Decrements the available count of a book if copies remain. Records the action in the undo Stack. If no copies are available, the student's name is enqueued in the waiting Queue.
Return Book	Increments the available count of a book, provided it does not already exceed the total quantity, protecting against inconsistent states.
View Issue Queue	Displays the FIFO waiting queue of students pending book issuance, formatted in a positional table.
Undo Last Action	Pops the most recent action from the undo Stack. If the action was a Delete, it reconstructs and re-inserts the book into both data structures. If it was an Issue, it reverses the availability decrement.
Statistics	Traverses the Linked List to compute: total unique titles, total physical copies, available copies, and currently issued copies. Derived in a single $O(N)$ pass.
Input Validation	All integer inputs are protected by a retry loop using <code>cin.clear()</code> and <code>cin.ignore()</code> to flush bad state. String inputs correctly use <code>getline</code> after <code>cin >></code> to avoid newline contamination.
Pre-loaded Data	Three seed books are inserted at startup (CLRS Algorithms, Clean Code, Design Patterns) for immediate demonstration without manual data entry.

3. Software Requirements

Requirement	Detail
Operating System	Linux (Ubuntu 20.04+), macOS 11+, or Windows 10+ with MinGW/WSL
Compiler	GCC 9.0+ / Clang 11+ (ISO C++17 required)
IDE (Optional)	VS Code with C/C++ extension, CLion, Code::Blocks, or plain terminal
Language Standard	ISO C++17 (uses structured bindings, <code>string_view</code> -compatible STL)
Standard Library Headers	<code><iostream></code> — I/O streams
	<code><string></code> — <code>std::string</code> for book attributes and names
	<code><iomanip></code> — <code>std::setw</code> , <code>std::left</code> for formatted console output
Runtime Dependencies	None — zero external dependencies; pure standard C++
Compilation Command	<code>g++ -std=c++17 -Wall -o lms lms.cpp</code>

4. Project Architecture

The system follows a **layered monolithic architecture** contained within a single translation unit (`lms.cpp`). Responsibility is divided into three clearly delineated zones:

4.1 Data Layer — Structs & Node Definitions (Lines 12–61)

This layer defines the raw data entities and the pointer-linked node types that back each data structure. No logic resides here — these are pure data carriers.

- **Book** — The central entity record (id, name, author, category, quantity, available).
- **ListNode** — Wraps a Book value and holds a next pointer for Linked List chaining.
- **BSTNode** — Holds a Book* pointer (alias into the Linked List heap object) and left/right children.
- **QueueNode** — Stores a requested book ID and student name, with a next pointer.
- **StackNode** — Polymorphic node (via two constructors and an ActionType enum) storing either a full Book copy (for delete-undo) or just a book ID (for issue-undo).

4.2 Data Structure Layer — Class Wrappers (Lines 67–160)

Two standalone classes encapsulate the Queue and Stack with clean public interfaces, destructors for RAII memory cleanup, and appropriately constrained access modifiers.

- **IssueQueue** — Manages front/rear pointers for FIFO enqueue/dequeue. Exposes: enqueue(), dequeue(), isEmpty(), displayQueue().
- **ActionStack** — Manages a top pointer for LIFO pushDelete()/pushIssue()/pop(). Throws `std::runtime_error` on empty pop.

4.3 Business Logic Layer — LibraryManager (Lines 166–520)

The **LibraryManager** class is the system controller. It owns the Linked List head pointer, the BST root pointer, an IssueQueue instance, and an ActionStack instance. All four private BST helper methods (`insertBST`, `searchBST`, `minValueNode`, `removeBST`) are encapsulated here, preventing external BST manipulation. Public methods map directly to the 10 menu operations plus `displayStatistics`.

4.4 Presentation Layer — main() & displayHeader() (Lines 526–583)

The entry point instantiates a single LibraryManager, seeds three books, then enters a do-while dispatch loop. The `displayHeader()` function renders the application banner. A switch-case dispatches user choices to LibraryManager methods.

4.5 Inter-Structure Communication

The BST does NOT store its own copy of each Book. `BSTNode::bookPtr` points directly to the Book object embedded in its `ListNode` on the heap. This creates a two-structure index over a single set of objects — mutations via BST (e.g., `updateBook`, `issueBook`) are immediately visible through the Linked List display, and vice versa. This is the most architecturally significant design decision in the system.

The consequence: when a book is deleted, the Linked List node must be deleted first (or simultaneously), and the BST node must be removed before the pointed-to Book is freed. The `deleteBook()` method respects this order — it saves a full copy to the stack, removes the BST node, then removes the Linked List node.

5. Program Flow

Stage	Description
Program Start	<code>main()</code> is entered. A <code>LibraryManager</code> object is constructed on the stack, initializing <code>head = nullptr</code> and <code>root = nullptr</code> . <code>IssueQueue</code> and <code>ActionStack</code> member objects are also default-constructed.
Initialization	Three seed books (IDs 101, 102, 103) are inserted via <code>addBook()</code> . Each insertion: (a) verifies ID uniqueness via <code>searchBST</code> , (b) allocates a <code>ListNode</code> on the heap, (c) front-inserts into the Linked List, (d) calls <code>insertBST</code> to add a <code>BSTNode</code> pointing to the new <code>ListNode</code> 's <code>Book</code> object.
Main Loop Entry	A do-while loop begins. <code>displayHeader()</code> prints the ASCII banner. The menu is printed. The system reads an integer choice with inline validation.
User Input	The integer is read via <code>cin >> choice</code> . An inline while loop guards against non-integer input by calling <code>cin.clear()</code> and <code>cin.ignore(10000, '\n')</code> before re-prompting.
Dispatch	A switch-case forwards the validated integer to the corresponding <code>LibraryManager</code> method. An invalid choice falls to the default case, which prints an error without breaking the loop.
Processing	Within each method, the operation reads further user input, performs data structure operations, updates state, and prints results with <code>[+]</code> , <code>[-]</code> , <code>[!]</code> , or <code>[✓]</code> prefix indicators.
Output	Results are printed to <code>stdout</code> using <code>cout</code> with <code>setw()</code> formatting where tabular layout is needed.
Loop Continuation	Control returns to the top of the do-while. The loop repeats until <code>choice == 11</code> (Exit).
Program Exit	The do-while exits. <code>main()</code> returns 0. The <code>LibraryManager</code> destructor fires: traverses and deletes all <code>ListNode</code> s, then calls <code>destroyBST()</code> to recursively delete all <code>BSTNode</code> s. <code>IssueQueue</code> and <code>ActionStack</code> destructors also fire, freeing their respective node chains.

6. Data Structures Used

6.1 Singly Linked List — Primary Book Store

Property	Detail
Node Type	struct ListNode { Book book; ListNode* next; }
Head Pointer	LibraryManager::head (private)
Insertion Strategy	Front insertion — $O(1)$. New node's next = head; head = newNode.
Traversal	Linear, head $\rightarrow$ tail via temp pointer — $O(N)$
Deletion	$O(N)$ traversal to predecessor node, then pointer re-linking and delete
Purpose	Canonical storage of all Book objects. Acts as the allocating owner of Book memory.
Time Complexity	Insert $O(1)$ Search $O(N)$ Delete $O(N)$ Display $O(N)$
Space Complexity	$O(N)$ — one ListNode per book

Why Linked List? Dynamic size without reallocation overhead. Supports $O(1)$ insertion at head. Acts as the memory owner for all Book data — BST nodes simply reference into it. A contiguous array would require costly reallocation or impose a fixed size limit.

6.2 Binary Search Tree (BST) — ID Search Index

Property	Detail
Node Type	struct BSTNode { Book* bookPtr; BSTNode* left; BSTNode* right; }
Root Pointer	LibraryManager::root (private)
Ordering Property	BST ordered on Book::id (integer)
Insertion	Recursive — $O(\log N)$ average, $O(N)$ worst (unbalanced)
Search	Recursive divide-and-conquer on ID — $O(\log N)$ average
Deletion	Classic 3-case BST deletion: no children, one child, two children (inorder successor strategy)
Pointer Ownership	BSTNode::bookPtr points INTO the Linked List heap — does NOT duplicate data
Time Complexity	Search/Insert/Delete $O(\log N)$ avg $O(N)$ worst (sorted insertion)
Space Complexity	$O(N)$ — one BSTNode per book, plus $O(\log N)$ recursion stack

Why BST? Provides $O(\log N)$ lookup by ID, which is far superior to the $O(N)$ linear scan of the Linked List for ID-based operations (updateBook, deleteBook, issueBook, returnBook). The tradeoff is that insertion order must maintain BST invariant, which the insertBST helper manages recursively.

6.3 Singly Linked Queue (IssueQueue) — Student Waitlist

Property	Detail
Node Type	struct QueueNode { int requestedBookId; string studentName; QueueNode* next; }
Pointers	front (dequeue end), rear (enqueue end)
Enqueue	Append to rear — $O(1)$
Dequeue	Remove from front — $O(1)$
Display	Traverse front $\rightarrow$ rear — $O(N)$
Purpose	Maintains FIFO ordering of students waiting for unavailable books
Time Complexity	All operations $O(1)$ except display $O(N)$
Space Complexity	$O(Q)$ where Q = number of pending requests

Why Queue? FIFO semantics are the fairest policy for waitlist management — the first student to request a book receives it first when stock is replenished. A Stack or random structure would violate fairness.

6.4 Singly Linked Stack (ActionStack) — Undo Mechanism

Property	Detail
Node Type	struct StackNode { ActionType action; Book savedBook; int issuedBookId; StackNode* next; }
Pointer	top (always points to most recent action)
Push Variants	pushDelete(Book b) — stores full Book; pushIssue(int id) — stores ID only
Pop	Returns StackNode by value, deallocates node — $O(1)$
ActionType Enum	UNDO_DELETE (0), UNDO_ISSUE (1)
Purpose	Records reversible operations in LIFO order for the undo feature
Time Complexity	Push $O(1)$ Pop $O(1)$
Space Complexity	$O(S)$ where S = number of undoable actions recorded

Why Stack? Undo semantics are inherently LIFO — the most recent action is always the one to reverse. A Stack models this precisely. The enum-based StackNode allows a single stack to multiplex two types of undo operations without requiring separate stacks.

7. Class Documentation

7.1 class IssueQueue

Property	Detail
Purpose	Encapsulates a dynamically linked FIFO queue for student book-issue requests.
Private Attributes	QueueNode* front — points to the dequeue-end node; QueueNode* rear — points to the enqueue-end node
Constructor	IssueQueue() : front(nullptr), rear(nullptr) {} — initializes empty queue
Destructor	~IssueQueue() — calls dequeue() in a while(!isEmpty()) loop to free all heap-allocated QueueNodes
enqueue(int, string)	Public. Allocates a new QueueNode and appends it at rear. Handles empty-queue edge case by setting both front and rear to the new node.
dequeue()	Public. Removes and deletes the front node. Re-sets rear to nullptr if the queue becomes empty after removal.
isEmpty() const	Public. Returns front == nullptr. const-qualified because it does not modify state.
displayQueue() const	Public. Traverses from front to nullptr, printing positional index, book ID, and student name in a formatted table.
Relationships	Owned by LibraryManager as a member object. LibraryManager calls enqueue() during issueBook() and displayQueue() during viewQueue().

7.2 class ActionStack

Property	Detail
Purpose	Encapsulates a dynamically linked LIFO stack recording reversible book operations.
Private Attributes	StackNode* top — points to the most recently pushed node
Constructor	ActionStack() : top(nullptr) {}
Destructor	~ActionStack() — calls pop() in a while(!isEmpty()) loop
pushDelete(Book b)	Public. Calls the StackNode(Book) constructor, setting action = UNDO_DELETE and storing the full Book by value. Prepends at top.
pushIssue(int id)	Public. Calls StackNode(int) constructor, setting action = UNDO_ISSUE and storing only the book ID. Prepends at top.
pop()	Public. Throws std::runtime_error if empty. Copies top's data into a local StackNode (by value), advances top, deletes the old node, and returns the copy.
isEmpty() const	Public. Returns top == nullptr.

Relationships	Owned by LibraryManager. deleteBook() calls pushDelete(); issueBook() calls pushIssue(); undoAction() calls pop().
---------------	--

7.3 class LibraryManager

Property	Detail
Purpose	Central business logic controller. Owns and manages all four data structures and implements all 10 library operations.
Private Attributes	ListNode* head — Linked List head; BSTNode* root — BST root; IssueQueue queue; ActionStack stack
Private Methods	insertBST, searchBST, minValueNode, removeBST, getValidInt — all internal helpers
Public Methods	addBook, menuAddBook, displayBooks, searchBook, updateBook, deleteBook, issueBook, returnBook, viewQueue, undoAction, displayStatistics, destroyBST
Constructor	LibraryManager() : head(nullptr), root(nullptr) {}
Destructor	Traverses and deletes all ListNodes. Calls destroyBST(root) to delete BSTNodes. Member destructors (IssueQueue, ActionStack) fire automatically.
Access Design	All BST operations are private — external code cannot directly manipulate the BST. Only LibraryManager's public API exposes safe, validated operations.
Relationships	Aggregates IssueQueue and ActionStack by value. Points to ListNode and BSTNode chains on the heap. BSTNode::bookPtr aliases into ListNode heap objects.

8. Function Documentation

Function	Access	Purpose & Logic	Time	Space
insertBST(BSTNode*, Book*)	Private	Recursively inserts a new BSTNode for the given Book* into the BST. Returns updated subtree root. Called by addBook() after front-inserting into the Linked List.	O(log N) avg	O(log N) stack
searchBST(BSTNode*, int id)	Private	Recursively searches for a BSTNode whose bookPtr->id matches id. Returns matching node or nullptr. Called by addBook (duplicate check), updateBook, deleteBook, issueBook, returnBook, undoAction.	O(log N) avg	O(log N) stack
minValueNode(BSTNode*)	Private	Iteratively walks leftward to find the BST node with the minimum ID in a given subtree. Used by removeBST for two-child deletion (inorder successor).	O(h) where h=height	O(1)
removeBST(BSTNode*, int id)	Private	Recursively locates the node with the given ID and removes it using the 3-case BST deletion algorithm. Returns updated subtree root. Called by deleteBook().	O(log N) avg	O(log N) stack
getValidInt()	Private	Reads an integer from cin. If cin enters a fail state (non-numeric input), clears error flags, flushes the stream, re-prompts, and retries indefinitely. Called whenever an integer input is required.	O(1) amortized	O(1)
addBook(int, string, string, string, int, int)	Public	Validates ID uniqueness, constructs a Book value, allocates a ListNode, front-inserts into LL, then calls insertBST. Called by menuAddBook() and undoAction().	O(log N)	O(1) new nodes
menuAddBook()	Public	Interactive wrapper. Reads user input for all Book fields, then delegates to addBook(). Separates UI from logic.	O(log N)	O(1)
displayBooks()	Public	Traverses the full Linked List from head to tail. Uses setw() for column-aligned tabular output. Truncates long strings to prevent column overflow.	O(N)	O(1)
searchBook()	Public	Presents choice of search mode. ID-search delegates to searchBST (O(log N)). Name-search performs LL linear scan (O(N)).	O(log N) or O(N)	O(1)

updateBook()	Public	Searches by ID via BST. Since BSTNode::bookPtr points directly into the LL's heap object, writing to node->bookPtr->name or author updates the canonical record in place.	O(log N)	O(1)
deleteBook()	Public	Saves full Book copy to ActionStack. Removes BSTNode. Then removes ListNode (head-case and mid/tail-case handled separately). Keeps both structures consistent.	O(log N) + O(N) LL delete	O(1)
issueBook()	Public	Searches by ID. If available > 0, decrements and pushes to ActionStack. Else enqueues student name in IssueQueue.	O(log N)	O(1)
returnBook()	Public	Searches by ID. Increments available if it is less than quantity (guards against over-return).	O(log N)	O(1)
viewQueue()	Public	Delegates to IssueQueue::displayQueue(). O(Q) where Q = queue length.	O(Q)	O(1)
undoAction()	Public	Pops ActionStack. If UNDO_DELETE: calls addBook() to reconstruct the book in both structures. If UNDO_ISSUE: searches by ID and increments available.	O(log N)	O(1)
displayStatistics()	Public	Single O(N) pass over LL accumulating totalTitles, totalCopies, availableCopies. Computes issuedCopies = totalCopies - availableCopies.	O(N)	O(1)
destroyBST(BSTNode*)	Public	Post-order recursive deletion of all BSTNodes (left, right, then self). Called by LibraryManager destructor.	O(N)	O(N) stack
displayHeader()	Free Function	Prints the fixed ASCII application banner. Stateless, no parameters.	O(1)	O(1)
main()	Free Function	Constructs LibraryManager, seeds 3 books, enters do-while menu loop with switch-case dispatch, exits on choice 11.	O(N log N) init	O(1)

9. Logic Explanation

9.1 Dual-Structure Insertion (addBook)

When `addBook()` runs, it first performs a BST search to confirm the ID does not already exist. If the ID is unique, it constructs a `Book` value on the stack, then allocates a `ListNode` on the heap using `new ListNode(newBook)`. The node is inserted at the head of the Linked List in $O(1)$ time (`newNode->next = head; head = newNode;`). Immediately after, `insertBST(root, &(head->book))` is called — crucially passing the address of the `Book` inside the newly inserted `ListNode`, not a local variable. This ensures the BST's pointer is stable for the lifetime of the `ListNode`. Every subsequent operation that navigates to a `BSTNode` and dereferences `bookPtr` is reading the canonical record, guaranteeing that a BST update and a LL display show identical data.

9.2 BST Recursive Search (searchBST)

The function applies the BST ordering invariant: if the target id is less than the current node's ID, recurse left; if greater, recurse right; if equal or node is `nullptr`, return the current node. The base cases ensure that a `nullptr` return signals 'not found' while a matching node returns itself. This binary halving of the search space yields $O(\log N)$ average time on a balanced tree — far superior to the $O(N)$ linear scan the Linked List would require for ID lookups.

9.3 BST Three-Case Deletion (removeBST)

Deleting from a BST must preserve the ordering invariant. `removeBST` handles three cases:

1. **No children:** the node is deleted and `nullptr` is returned to the parent, severing the link.
2. **One child:** the node is deleted and its sole child pointer is returned to the parent, effectively splicing out the deleted node.
3. **Two children:** The most complex case. The inorder successor — the smallest node in the right subtree — is found via `minValueNode()`. This successor's `bookPtr` is copied into the current node (conceptually replacing its value), then the successor itself is deleted from the right subtree via a recursive call. This preserves BST ordering because the inorder successor is always greater than everything in the left subtree and less than everything in the right subtree above it.

Critical note on pointer management: when the two-children case executes `node->bookPtr = temp->bookPtr`, it is reassigning the pointer stored inside a `BSTNode` to point to a different `Book` object in the Linked List. The `BSTNode` structure is preserved; only its `bookPtr` is redirected. The original `BSTNode` is NOT deleted here — only its index pointer is updated.

9.4 Delete Synchronization (deleteBook)

`deleteBook()` must keep both the Linked List and BST consistent. The sequence is: (1) search BST to confirm existence and obtain the `bookPtr` for stack saving; (2) push a full **copy** of the `Book` value onto the `ActionStack` (critical — the copy is made before the Linked List node is freed); (3) remove the `BSTNode` via `removeBST` (this detaches the index but does not free the LL node yet); (4) remove the `ListNode` from the Linked List, handling the head-node edge case separately from interior/tail nodes. Only after step 4 is the heap memory freed.

9.5 Undo Logic (undoAction)

The undo stack is polymorphic in its node content. When popped, the `ActionType` enum field is inspected. For `UNDO_DELETE`: the saved `Book` value (a full copy preserved before deletion) is passed to `addBook()`, which reconstructs both the Linked List node and the BST entry. For `UNDO_ISSUE`: `searchBST` locates the book by its stored ID and increments `available`. The single stack handles both operation types without duplication of stack infrastructure.

9.6 Queue Enqueue / Dequeue Logic

IssueQueue uses a front/rear dual-pointer pattern. On enqueue: if `rear == nullptr` (empty queue), both front and rear point to the new node. Otherwise, `rear->next` is set to the new node and rear advances. On dequeue: front advances to `front->next`, and if the result is `nullptr` (last node removed), rear is also set to `nullptr`. This prevents a dangling rear pointer on empty queue, which is a common bug in singly-linked queue implementations.

9.7 Input Validation (getValidInt)

When `cin >> x` fails due to non-numeric input, the stream enters a fail state and subsequent reads silently skip. The validation loop calls `cin.clear()` to clear the error flags, then `cin.ignore(10000, '\n')` to discard all characters in the buffer up to the next newline. This restores the stream to a usable state before re-prompting. After a successful `cin >> x`, `cin.ignore(10000, '\n')` is called once more to consume the trailing newline, preventing it from being picked up by subsequent `getline()` calls.

10. Code Walkthrough

10.1 Header Inclusions (Lines 1–5)

`#include <iostream>` — Provides `cin`, `cout`, `cerr` for all console I/O.

`#include <string>` — Provides `std::string` for book title, author, category, and student name storage.

`#include <iomanip>` — Provides `std::setw()` and `std::left` for the column-aligned table output in `displayBooks()` and `displayQueue()`.

`using namespace std;` — Imports the entire `std` namespace. Acceptable for single-file educational projects; in production code, explicit qualification (`std::string`) is preferred.

10.2 Struct Definitions (Lines 12–61)

Book (12–19): Six fields covering the unique identifier, descriptive metadata, and inventory state. The separation of quantity (total copies ever added) from available (copies currently in library) is the key inventory accounting model.

ListNode (22–27): Stores a `Book` by value — meaning the `ListNode` owns its `Book`. Its one-argument constructor uses a member initializer list (`: book(b), next(nullptr)`) for efficient initialization. The owned `Book`'s address (`&head->book`) is passed to BST insertion.

BSTNode (30–36): Stores a raw non-owning pointer to `Book`. The constructor sets `bookPtr`, `left`, and `right` via initializer list. The semantic contract is that `BSTNode` never allocates or frees a `Book` — it only indexes one.

QueueNode (39–45): Stores both the book ID being requested and the student name. The constructor accepts `const string&` to avoid copying the name during node creation.

StackNode (50–61): The `ActionType` enum (`UNDO_DELETE`, `UNDO_ISSUE`) makes the node self-describing. Two overloaded constructors enable type-safe creation: the `Book`-taking constructor sets `action = UNDO_DELETE` and copies the full `Book`; the `int`-taking constructor sets `action = UNDO_ISSUE` and stores only the ID. The unused field in each case takes a default value (`issuedBookId = -1` for delete nodes).

10.3 IssueQueue Class (Lines 67–119)

Constructor and destructor provide RAII guarantees — no `QueueNode` can outlive the `IssueQueue` object. The `enqueue()` method's empty-queue branch (setting both front and rear) prevents the common mistake of only updating rear, which would leave front dangling. `displayQueue()` uses a local temp pointer so it never mutates front or rear during traversal.

10.4 ActionStack Class (Lines 121–160)

`pushDelete()` and `pushIssue()` use constructor overloading to disambiguate the two node types. The `pop()` method returns `StackNode` by value — it copies the data out before deleting the heap node, ensuring the returned data is valid. The `throw runtime_error` on empty `pop()` provides a safety net, though `undoAction()` guards against empty stack before calling `pop()`.

10.5 LibraryManager Private BST Helpers (Lines 174–237)

All four BST helpers are recursive and follow the standard textbook BST algorithms. They are private because no external code should interact with BST nodes directly — the LibraryManager's public API provides the only safe entry points. `getValidInt()` is also private because input handling is an internal concern of the class.

10.6 LibraryManager Destructor (Lines 242–251)

The destructor traverses the LL: saves `current->next` in a local variable before deleting `current` to avoid use-after-free. After all `ListNodes` are deleted (and their embedded `Books` freed), `destroyBST(root)` is called. The order matters: `BSTNodes` only hold pointers into the now-freed `Books`. If `destroyBST` were called first, the BST nodes would be freed but the `Book` heap objects (in LL nodes) would remain — a memory leak. Freeing LL nodes first, then BST nodes, is the correct order.

10.7 main() (Lines 534–583)

`LibraryManager` library is declared on the stack — its lifetime is the scope of `main()`. Three seed calls to `addBook()` pre-populate the system. The `do-while` ensures the menu is displayed at least once before the exit condition is checked. The `cin >>` choice with its inline validation loop exactly mirrors `getValidInt()` but is inlined in `main()` because it is outside the class. A separate call to `cin.ignore()` after the successful read prevents newline carry-over into subsequent `getline()` calls within dispatched methods.

11. Object-Oriented Programming Concepts

Concept	Application in This Project
Encapsulation	<code>LibraryManager</code> hides <code>head</code> , <code>root</code> , <code>queue</code> , <code>stack</code> , and all BST helper methods as private. External code cannot directly manipulate any data structure — it must go through the validated public API. <code>IssueQueue</code> and <code>ActionStack</code> similarly hide their <code>front/rear</code> and <code>top</code> pointers.
Abstraction	The caller of <code>issueBook()</code> does not know whether it interacts with a BST or a Queue internally — the complexity is hidden behind a single method name. Similarly, <code>searchBook()</code> abstracts two entirely different search algorithms behind one interface.
Constructors	Every struct and class uses constructor initializer lists for efficient member initialization. <code>ActionStack</code> uses two overloaded constructors (for <code>Book</code> and <code>int</code>) to support polymorphic undo nodes.
Destructors	<code>IssueQueue</code> , <code>ActionStack</code> , and <code>LibraryManager</code> all define custom destructors to free heap memory. This is necessary because the objects own heap-allocated nodes that the compiler cannot automatically reclaim.
Object Composition	<code>LibraryManager</code> composes <code>IssueQueue</code> and <code>ActionStack</code> as value members (not pointers). Their lifetimes are tied to <code>LibraryManager</code> — they are constructed and destroyed automatically.
Const Correctness	<code>isEmpty()</code> is <code>const</code> -qualified in both <code>IssueQueue</code> and <code>ActionStack</code> . <code>displayQueue()</code> is <code>const</code> -qualified. These guarantee that calling code can invoke them on <code>const</code> references without triggering compilation errors.
Access Specifiers	<code>private</code> : separates internal implementation details (BST helpers, raw pointers) from <code>public</code> : (the 10 library operations). This enforces the

	open-closed principle within the class boundary.
Method Overloading	StackNode has two constructors with different signatures — StackNode(Book b) and StackNode(int id) — demonstrating constructor overloading used purposefully for semantic clarity.
Polymorphism (Ad-hoc)	The undoAction() method inspects the ActionType enum at runtime to decide which reversal path to take — a form of runtime dispatching without virtual functions, appropriate for a closed set of action types.

12. Algorithms Used

Algorithm	Working	Complexity	Advantage
BST Insertion (insertBST)	Compares target ID with current node. If smaller, recurse left; if larger, recurse right; if null, allocate and return new node. The return value propagates upward to re-link the tree.	$O(\log N)$ avg $O(N)$ worst	Maintains sorted index; enables binary search lookups
BST Search (searchBST)	Navigates the tree by comparing target ID with each node. Binary halving eliminates half the remaining candidates at each level.	$O(\log N)$ avg	Significantly faster than linear scan for large datasets
BST Deletion — Inorder Successor	For two-child deletion, finds the minimum node in the right subtree (leftmost node). Copies its bookPtr into the node to delete, then recursively deletes the successor from the right subtree.	$O(\log N)$ avg	Preserves BST invariant without restructuring the entire tree
Linked List Traversal	Iterates temp = temp->next from head until nullptr for display, statistics, and name search.	$O(N)$	Simplest traversal; no auxiliary structures needed
FIFO Queue Scheduling	Students are served in the order they arrived (first-in, first-out) when book stock is replenished.	$O(1)$ per op	Fair scheduling; deterministic ordering
LIFO Stack Undo	The most recently recorded action is always undone first, matching user expectations for undo behavior.	$O(1)$ per op	Natural temporal ordering for reversal operations

13. Error Handling

Error Scenario	Handling Strategy
Non-integer input at menu or ID prompt	<code>getValidInt()</code> and the inline <code>main()</code> loop both use <code>while(!(cin >> x))</code> with <code>cin.clear() + cin.ignore(10000, '\n')</code> to recover from stream fail state and re-prompt without crashing.
Duplicate Book ID on add	<code>searchBST(root, id)</code> is called before insertion. If a matching node is found, the method prints an error and returns early without modifying any data structure.
Book not found (search/update/delete/issue/return)	BST search returns <code>nullptr</code> . The calling method checks for <code>nullptr</code> and prints a '['!]' message, then returns. No operation is performed on a <code>nullptr</code> .
Issuing an out-of-stock book	<code>issueBook()</code> checks <code>available > 0</code> . If false, it offers the waitlist option via <code>IssueQueue::enqueue()</code> instead of attempting to decrement below zero.
Returning a book when all copies are in	<code>returnBook()</code> checks <code>available < quantity</code> before incrementing. Prevents the available count from exceeding total quantity.
Undo when stack is empty	<code>undoAction()</code> calls <code>stack.isEmpty()</code> before popping. If empty, prints '['!]' No actions to undo.' and returns. Avoids the <code>runtime_error</code> that <code>pop()</code> would throw.
Displaying books when library is empty	<code>displayBooks()</code> checks <code>head == nullptr</code> and prints a message before attempting any traversal.
Displaying queue when queue is empty	<code>IssueQueue::displayQueue()</code> checks <code>isEmpty()</code> and prints a message before traversal.
Delete on empty Linked List	Implicitly prevented: <code>searchBST</code> would return <code>nullptr</code> first, causing early return before any LL pointer arithmetic.
Stack empty pop (safety net)	<code>ActionStack::pop()</code> throws <code>std::runtime_error</code> if called on empty stack. This is a defensive fallback; all callers already guard via <code>isEmpty()</code> .

14. Memory Management

14.1 Dynamic Allocation Points

Allocation Site	Object Allocated
new ListNode(newBook) in addBook()	One ListNode per book inserted
new BSTNode(b) in insertBST()	One BSTNode per book inserted
new QueueNode(bookId, studentName) in enqueue()	One QueueNode per waitlist entry
new StackNode(b) in pushDelete()	One StackNode per delete operation recorded
new StackNode(id) in pushIssue()	One StackNode per issue operation recorded

14.2 Deallocation Sites

Deallocation Site	What Is Freed
LibraryManager destructor LL loop	All ListNodes (and their embedded Book objects)
destroyBST() post-order recursion	All BSTNodes (not the Books they pointed to — those were freed in LL)
IssueQueue destructor while loop	All QueueNodes
ActionStack destructor while loop	All StackNodes
deleteBook() — delete temp (LL node)	The specific ListNode being deleted mid-session
removeBST() — delete node	The specific BSTNode being removed mid-session
IssueQueue::dequeue() — delete temp	One QueueNode at a time (not currently used by any operation, but structurally available)
ActionStack::pop() — delete temp	One StackNode per undo action

14.3 Known Limitations & Suggestions

- The BST is not self-balancing. Inserting books with sequential IDs (101, 102, 103 as in the seed) degrades BST to $O(N)$ due to right-skewing. Replacing with an AVL tree or Red-Black tree would guarantee $O(\log N)$ worst-case.
- The ActionStack grows indefinitely and is never pruned. In a long session with many operations, this could consume significant heap memory. A maximum stack depth limit or a periodic truncation policy would improve production safety.

- The IssueQueue dequeue() is never called automatically when a book is returned. The system records waitlist entries but does not auto-process them on returnBook(). This is a feature gap, not a memory hazard.
- No memory leak exists at normal exit: all four custom destructors clean up fully. Verified: LL loop, destroyBST recursion, IssueQueue destructor, ActionStack destructor.
- Potential leak on abnormal exit (e.g., SIGKILL): OS reclaims all process memory, so this is not a practical concern in this context.

15. User Interface

15.1 Console Layout

The application is entirely console-based. Each screen cycle consists of:

4. The ASCII banner header (displayHeader) printed by a single function — clean separation of presentation.
5. A numbered menu list (1–11) with two-space indentation.
6. A separator line and a prompt.
7. The selected operation's output, prefixed with one of four status indicators:

[+] — Success, data added or updated.

[-] — Success, data removed.

[!] — Warning or error, no state change.

[✓] — Confirmation, undo or search success.

15.2 Formatted Table Output

displayBooks() uses `cout << left << setw(N)` to produce six aligned columns: ID (10), Title (30, truncated at 28), Author (20, truncated at 18), Category (15, truncated at 13), Total (10), Available (10). The 95-character divider matches the total column width, keeping the table visually clean regardless of terminal width.

15.3 Input Design

Integer inputs (IDs, quantities, menu choices) always go through the `getValidInt()` retry loop. String inputs (title, author, student name) use `getline()` exclusively after any preceding `cin >>` to avoid newline contamination — a common pitfall with mixed-mode input in C++.

16. Diagram Suggestions

The following diagram placements are recommended. All diagrams should be created externally (draw.io, Lucidchart, PlantUML) and inserted at the indicated positions.

Figure 1: System Architecture Diagram

Property	Detail
Placement	Section 4 (Project Architecture), after §4.5
Description	Shows the three-layer architecture (Data Layer, Data Structure Layer, Business Logic Layer, Presentation Layer) and how they communicate. Boxes for each layer with directional arrows.

Figure 2: Class Diagram

Property	Detail
Placement	Section 7 (Class Documentation), after the LibraryManager table
Description	Shows struct Book, struct ListNode, struct BSTNode, struct QueueNode, struct StackNode, class IssueQueue, class ActionStack, class LibraryManager with attributes, methods, access specifiers, and composition/dependency arrows. Include the pointer relationship BSTNode → Book (inside ListNode).

Figure 3: Program Flowchart

Property	Detail
Placement	Section 5 (Program Flow), after the flow table
Description	Start node → Constructor → Seed Data → do-while Loop → displayHeader → Read Choice → switch → [11 operation branches] → Exit node. Each branch shows the sub-steps of that operation.

Figure 4: Use Case Diagram

Property	Detail
Placement	Section 3 (Project Overview), after §1.3
Description	Actor: User (Librarian/Student). Use Cases: Add Book, Search Book, Update Book, Delete Book, Issue Book, Return Book, View Queue, Undo, View Statistics, Exit. Include <<extend>> from Issue to Waitlist for the out-of-stock path.

Figure 5: BST State Diagram

Property	Detail
Placement	Section 6.2 (BST Data Structure)
Description	Show BST state after inserting IDs 103, 101, 102 (the seed data). 103 as root, 101 as left child, 102 as right child of 101. Illustrate the search path to find ID 101 vs 102.

Figure 6: Linked List Memory Diagram

Property	Detail
Placement	Section 6.1 (Linked List)
Description	Draw heap boxes for ListNode(103), ListNode(101), ListNode(102) with forward arrows and a head pointer. Show BSTNode pointers aliasing into the embedded Book fields.

Figure 7: Queue State Diagram

Property	Detail
Placement	Section 6.3 (Queue)
Description	Show front and rear pointers. Demonstrate enqueue of 'Ali' for Book 103, then 'Sara' for Book 103. Then show state after one dequeue.

Figure 8: Stack State Diagram

Property	Detail
Placement	Section 6.4 (Stack)
Description	Show top pointer with stacked StackNodes after: Issue Book 101, Delete Book 102, Undo (removes delete). Illustrate LIFO ordering.

Figure 9: Sequence Diagram — Delete + Undo

Property	Detail
Placement	Section 9.4 and 9.5 (Logic Explanation)
Description	Shows message flow: User → LibraryManager::deleteBook() → ActionStack::pushDelete() → removeBST() → LL removal → User → LibraryManager::undoAction() → ActionStack::pop() → addBook().

Figure 10: Data Flow Diagram (Level 0 — Context)

Property	Detail
Placement	Section 4 (Architecture)
Description	Single process bubble 'Library Management System'. External entity: User. Data flows in: Book Data, User Input. Data flows out: Book Records, Search Results, Statistics, Confirmations.

Figure 11: Data Flow Diagram (Level 1)

Property	Detail
Placement	Section 5 (Program Flow)
Description	Explode the Level 0 bubble into: Add/Update/Delete (write path via LL + BST), Search (read path via BST or LL), Issue/Return (availability management + Queue), Undo (Stack), Statistics (LL scan).

17. Screenshots Section

The following terminal screenshots are recommended. Capture them by running the compiled binary, performing the specified operations, and inserting the screenshot image at the indicated position.

Screenshot 1: Application Startup — Main Menu

Property	Detail
Placement & Caption	Section 5 (Program Flow) after the flow table. Caption: 'Figure 12: Main Menu — Showing the ASCII banner and 11-option numbered menu.'
How to Capture	Run the binary without selecting any option.

Screenshot 2: Add Book Operation

Property	Detail
Placement & Caption	Section 8 (Function Documentation) beside addBook entry. Caption: 'Figure 13: Add Book — Successful insertion with [+] confirmation.'
How to Capture	Add a new book with ID 201.

Screenshot 3: Display Books — Tabular Output

Property	Detail
Placement & Caption	Section 7.3 (LibraryManager — displayBooks). Caption: 'Figure 14: Display All Books — Formatted column-aligned table output.'
How to Capture	After adding 2–3 books, run option 2.

Screenshot 4: Search by ID (BST)

Property	Detail
Placement & Caption	Section 6.2 (BST). Caption: 'Figure 15: BST ID Search — $O(\log N)$ lookup with [✓] confirmation.'
How to Capture	Search for an existing ID and a non-existing ID.

Screenshot 5: Search by Name (Linked List)

Property	Detail
Placement & Caption	Section 6.1 (Linked List). Caption: 'Figure 16: Name Search — Linear scan result.'
How to Capture	Search for 'Clean Code'.

Screenshot 6: Issue Book — Success + Waitlist

Property	Detail
Placement & Caption	Section 8 (issueBook). Caption: 'Figure 17: Issue Book — Stock decremented; student added to queue on second issue when stock is 1.'
How to Capture	Issue the same book twice.

Screenshot 7: View Issue Queue

Property	Detail
Placement & Caption	Section 6.3 (Queue). Caption: 'Figure 18: Issue Queue — FIFO display of pending requests.'
How to Capture	View the queue after two waitlist entries.

Screenshot 8: Delete + Undo

Property	Detail
Placement & Caption	Section 9.4 & 9.5. Caption: 'Figure 19: Delete and Undo — Book removed then restored via LIFO undo.'
How to Capture	Delete a book, view books, then undo, view books again.

Screenshot 9: Statistics

Property	Detail
Placement & Caption	Section 8 (displayStatistics). Caption: 'Figure 20: Statistics Dashboard — Computed totals from single-pass LL traversal.'
How to Capture	Run option 10 after a mix of issues and returns.

Screenshot 10: Invalid Input Recovery

Property	Detail
Placement & Caption	Section 13 (Error Handling). Caption: 'Figure 21: Input Validation — Stream recovery loop prompting for re-entry.'
How to Capture	Type a letter when prompted for a number.

18. Code Quality Review

18.1 Strengths

- **Purposeful data structure selection:** Each structure is used where it genuinely provides the best asymptotic behavior for that operation — BST for ID lookup, LL for storage, Queue for FIFO waitlisting, Stack for LIFO undo. This is not cosmetic; it is architecturally sound.
- **Pointer aliasing architecture:** `BSTNode::bookPtr` pointing into LL heap objects eliminates data duplication and ensures consistency. Any mutation via BST is immediately reflected in LL traversal.
- **RAII-compliant destructors:** All three classes with heap resources define custom destructors that fully clean up their chains. No reliance on garbage collection; correct C++ manual memory management.
- **Input validation coverage:** `getValidInt()` provides a reusable guard for all integer input points. The follow-up `cin.ignore()` prevents `getline()` contamination.
- **Separation of concerns:** `menuAddBook()` separates UI I/O from the logic in `addBook()`. This enables `addBook()` to be reused programmatically (e.g., from `undoAction()`) without any console I/O side effects.
- **Status prefixes ([+], [-], [!], [✓]):** Consistent feedback indicators give the UI a professional, systematic appearance.
- **Self-commenting in-code labels:** Comments like `'// Insert into Linked List (Front insertion for O(1))'` and `'// Insert reference into BST (O(log N))'` demonstrate understanding of time complexity inline with implementation.

18.2 Weaknesses

- **No file persistence:** All data is lost on exit. A production system would serialize to a file (CSV, binary, or JSON) and deserialize on startup.
- **Unbalanced BST:** The seed data inserts IDs 101, 102, 103 in ascending order, creating a right-skewed tree. Searching for ID 103 requires visiting all three nodes — $O(N)$ behavior. An AVL tree would guarantee $O(\log N)$.
- **Name search is case-sensitive and exact:** Searching for 'clean code' when the stored value is 'Clean Code' returns no result. A real-world system would normalize case or use substring matching.
- **Undo queue not processed on return:** `returnBook()` increments available but does not dequeue the next waiting student or reduce their queue position.
- **No role-based access control:** All operations are equally accessible. A production system would restrict delete, update, and statistics to admin accounts.
- **using namespace std:** Acceptable in educational contexts; problematic in larger codebases due to name collision risks.
- **Long function bodies in LibraryManager:** `deleteBook()` performs BST removal, LL removal, and stack push within one function. Extracting the LL removal to a private `removeFromList(int id)` helper would improve modularity.

18.3 Naming Convention

The project uses camelCase for methods and local variables (`addBook`, `getValidInt`, `requestedBookId`), PascalCase for struct/class names (`Book`, `ListNode`, `LibraryManager`), and UPPER_SNAKE_CASE for enum values (`UNDO_DELETE`, `UNDO_ISSUE`). This is consistent throughout and aligns with common C++ style guides.

18.4 Readability & Maintainability

Section headers using `===` delimiters and numbered comment prefixes (`// 1. Add Book`, `// 2. Display Books`) make the file scannable. All critical design decisions (pointer aliasing, inorder successor, RAII) are explained in inline comments. The code is maintainable by any developer with intermediate C++ and data structures knowledge.

19. Optimization Suggestions

Optimization	Rationale
Replace BST with AVL Tree	The BST becomes $O(N)$ for sorted or near-sorted insertions (as with the seed IDs 101, 102, 103). An AVL tree maintains balance through rotations, guaranteeing $O(\log N)$ for all operations.
Add File Persistence	Serialize the Linked List to a CSV or binary file on exit; deserialize on startup. Without this, the system has no production utility. A simple ofstream/ifstream pass over the LL is sufficient.
Auto-process Waitlist on Return	returnBook() should call queue.displayQueue() or dequeue the front waiting student and issue them the returned copy automatically, closing the loop between Queue and inventory.
Case-insensitive Name Search	Convert both the query string and stored name to lowercase before comparison using std::transform. This makes name search usable in practice.
Limit ActionStack Depth	Cap the undo stack at $N=50$ entries. On overflow, drop the oldest entry (convert to a deque internally) to prevent unbounded heap growth in long sessions.
Extract removeFromList(int id) Helper	Pulling the Linked List removal logic into a private helper removes code duplication between deleteBook() and any future operation requiring LL node removal.
Add Partial / Substring Search	Currently name search requires exact match. A loop comparing using std::string::find() would return all books whose title contains the query substring, greatly improving usability.
Add Category Filter to Display	A displayBooks(string category) overload that filters the LL traversal to only matching books would be immediately useful for a real library.
Replace raw new/delete with smart pointers	Using std::unique_ptr<ListNode> and std::unique_ptr<BSTNode> would make memory ownership semantically explicit and eliminate the risk of forgetting to call delete in error paths.
Add Book ID to StackNode for Delete Undo	Currently deleteBook() pushes a full Book copy (including all fields). This is correct and safe but consumes more memory than necessary. A separate lookup via stored ID at undo time would reduce stack memory, though at the cost of an $O(\log N)$ search during undo.

20. Conclusion

Project Achievement Summary. The Library Management System successfully implements a console-based book management solution using four manually constructed data structures — Singly Linked List, Binary Search Tree, FIFO Queue, and LIFO Stack — each fulfilling a distinct and justified role within the system. All ten core operations (add, display, search by ID, search by name, update, delete, issue, return, view queue, undo, and statistics) are implemented with correct logic and consistently validated input handling.

Data Structures Competency. The project demonstrates a mature understanding of data structure selection principles: the BST is used not as an alternative to the Linked List but as a complementary index over it, eliminating redundant data while providing $O(\log N)$ ID lookups. The pointer aliasing architecture (`BSTNode::bookPtr` into LL heap objects) is a sophisticated design pattern that reflects production-grade thinking about single-source-of-truth data ownership.

OOP and Memory Safety. The system demonstrates encapsulation, abstraction, RAII-compliant destructors, const-correct methods, and constructor overloading. Memory is allocated and deallocated in a structured, traceable manner, with no identified memory leaks at normal exit.

Technical Skills Demonstrated. Recursive BST insertion, search, and three-case deletion; singly-linked FIFO queue with front/rear management; LIFO stack with polymorphic node types; stream-state-aware integer input validation; formatted tabular console output with `setw/iomanip`; and dual-structure synchronized delete/undo are all implemented correctly.

Future Scope. The next iteration of this system would benefit from: (1) file I/O persistence using `ofstream/ifstream`; (2) an AVL tree replacing the BST for guaranteed $O(\log N)$ balance; (3) a role-based access layer separating admin and student operations; (4) case-insensitive and partial-match name search; (5) automatic waitlist processing on book return; and (6) a C++ Standard Library migration replacing manual linked structures with `std::list`, `std::map`, `std::queue`, and `std::stack` to demonstrate fluency with the STL.

— End of Documentation —